

DSST389: Exam III — Reference Sheet

This sheet is attached to your exam. You do not need to memorize any of it.

Building a network. A network is a set of **nodes** and a set of **edges** joining pairs of them. It is recorded as an **edge list**: one row per edge, with the endpoints in the columns `doc_id` and `doc_id2`. In a **directed** network the two ends differ; in an **undirected** one they do not. Any directed network can be read as undirected by ignoring the direction; the reverse is not possible.

```
node, edge, G = DSNetwork.process(edges, directed=False)
```

- The graph is built *from the edge list*, so a node belonging to no edge is never created. Filtering an edge list deletes nodes, silently.
- **node** has one row per node: `id`, the layout coordinates `x` and `y`, and the metrics below. **edge** has one row per edge, with `x`, `y`, `xend`, `yend`, which are the four aesthetics of `geom_segment`. Add `arrow=arrow(length=0.02)` to draw direction.

Components. A **component** is a set of nodes all reachable from one another by following edges. A network with exactly one component is **connected**. The **component** column numbers them in **descending order of size**, so component 1 is always the largest, and **component_size** gives the size of each.

Centrality. All four scores below are computed **separately within each component**, because the last three are only defined on a connected network. Filter to `component == 1` before comparing scores across nodes.

- **degree** — the number of neighbors a node has. Purely local.
- **eigen** — *eigenvalue centrality*. A score proportional to the sum of the scores of its neighbors, so that being adjacent to central nodes makes a node central. **Rescaled so the largest score in each component is 1**, which means a score of 1 says “the most central node *in this component*” and cannot be compared across components.

$$s_j = \lambda \cdot \sum_{i \in \text{Neighbors}(j)} s_i$$

- **close** — *closeness centrality*. Take the shortest-path distance from the node to every other node in its component, and add up the **reciprocals**. A neighbor contributes 1, a node four hops away contributes only 1/4, so the score rewards being near *everything*. Undirected networks only.
- **between** — *betweenness centrality*. For every pair of other nodes, consider the shortest paths between them; count what fraction of them run *through* this node, and add that up over all pairs. High for **bridges**: nodes that are not popular but are unavoidable. Degree and betweenness can disagree completely about the same node.
- **cluster** — a community label, assigned by an algorithm that tries to put more edges inside groups than between them. It is an output, not a property: a node on the boundary between two groups may be assigned arbitrarily.

Directed networks. Setting `directed=True` changes the node table. **degree** is replaced by **degree_out** (links on the page), **degree_in** (links into the page), and **degree_total**. **eigen** and **between** are still computed but ignore the direction. **close is not available**, because a distance to a node you cannot reach is not defined.

Kinds of network.

- **Citation.** An edge whenever one page links to another. Sensitive to page length, so summarize by links *in* rather than links *out*.
- **Co-citation.** An edge between two pages whenever some *third* page cites **both**. Always undirected. The **count** column holds the number of citing pages a pair shares, and edges are usually kept only above a threshold. One page citing n items manufactures $n(n-1)/2$ co-citations at a stroke.
- **Distance.** An edge whenever two documents are closer than a cutoff, using the angle distances of Chapter 15.
- **Symmetric nearest neighbor.** Take each document’s k nearest neighbors, and keep an edge only when two documents are **mutually** in each other’s sets. No node can then exceed degree k , which stops popular documents from becoming hubs.

Images as arrays. `cv2.imread(path)` returns a NumPy array of shape (**height, width, channels**) with values from 0 to 255. `img.mean()` averages over **every number**, channels included: a 2×2 color image has 12 of them, not 4.

- **OpenCV stores channels in BGR order, not RGB.** `[255, 0, 0]` is *blue* and `[0, 0, 255]` is *red*. `[0,0,0]` is black and `[255,255,255]` is white.

- `cv2.cvtColor(img, cv2.COLOR_BGR2HSV)` — convert to hue, saturation, value. Same shape. OpenCV scales **hue to 0–179** (half of degrees, to fit in 8 bits) and saturation and value to 0–255.
- `DSImage.compute_colors(img_hsv)` — the proportion of pixels in each color category; the proportions sum to 1. It checks saturation and value *before* hue, so that dark and desaturated pixels are called black, white, or gray rather than being binned by a hue that means nothing.
- `DSImage.plot_image_grid(df, ncol=10, label_name="label")` — display images.

RGB to HSV. Given R, G, B scaled to $[0, 1]$, let $M = \max(R, G, B)$, $m = \min(R, G, B)$, and let the chroma be $C = M - m$. Then $V = M$, and:

$$S = \begin{cases} 0 & \text{if } V = 0 \\ \frac{C}{V} & \text{otherwise} \end{cases} \quad H = \begin{cases} 0^\circ & \text{if } C = 0 \\ 60^\circ \times \frac{G - B}{C} \bmod 360^\circ & \text{if } M = R \\ 60^\circ \times \left(\frac{B - R}{C} + 2 \right) & \text{if } M = G \\ 60^\circ \times \left(\frac{R - G}{C} + 4 \right) & \text{if } M = B \end{cases}$$

Hue is an angle on a color wheel: red 0° , yellow 60° , green 120° , cyan 180° , blue 240° , magenta 300° , wrapping back to red at 360° . When $C = 0$ the pixel is gray and **has no hue**; the formula would divide by zero, and the 0° it returns instead is indistinguishable from red.

Images as vectors.

- `ViTEEmbedder()` — returns a **dense** vector of **768** numbers for an image. Unlike a TF-IDF vector, nearly every entry is non-zero and no single dimension has a nameable meaning. Only the geometry matters.
- `umap(df, features=[c.vit]).predict(full=True)` — project to two dimensions, returned as the columns `dr0` and `dr1`.
- **Transfer learning:** reusing a model trained by someone else, on other images, for other labels. What transfers is the *representation*, not the labels.
- `SigLIPEmbedder()` — embeds **images and text into the same space**, with `.embed_image(path)` and `.embed_text(string)`. The vectors are unit length, so `dot_product(a, b)` is exactly the cosine similarity of Chapter 15: 1 means identical, 0 means orthogonal, and the angle distance is `arccos` of it.
- **Zero-shot classification:** embed each category description as text, and give each image the label whose text vector it has the largest dot product with. No training, no labeled examples. It is an *argmax over the candidate list you supplied*, so there is no “none of the above,” and adding a category can change a prediction that was already made.